

Case Study: Synthetic Speech Data Pipeline (S.S.D.P.)

Timebox: Up to 72 hours

Context

At Olimi AI, we build AI-first speech systems with a focus on Arabic, particularly Egyptian Arabic. A core part of the work is producing high-quality training data — including synthetic data — for fine-tuning STT models. The bottleneck is rarely the model; it's the data.

This case study assesses how you design and build a practical data pipeline that produces training-ready synthetic speech data.

Objective

Build a pipeline that produces a synthetic Egyptian Arabic speech dataset suitable for fine-tuning an STT model. The pipeline has four stages:

- Generate text prompts/scripts in Egyptian Arabic
- Synthesize audio from those prompts using TTS
- Provide a way to review the resulting (text, audio) pairs
- Prepare and export the reviewed data in a structured form ready for training

This exercise evaluates **data engineering judgment, attention to data quality, and practical Arabic/dialect awareness**. You do not need to train a model.

The Challenge

Design and implement the pipeline above. You choose the tools, models, sources, and formats at every stage.

Egyptian Arabic is the target dialect because it presents real challenges for off-the-shelf speech systems. Your pipeline should reflect awareness of those challenges — part of the exercise is identifying what they are.

Core Expectations

1. Prompt Generation

Produce a corpus of Egyptian Arabic text prompts to feed into TTS. Document your approach and rationale.

2. TTS Synthesis

Synthesize audio from the prompts. Document your choices and rationale.

3. Review

Provide a way to review the generated (text, audio) pairs. The goal is to be able to tell good samples from bad ones before they enter the training set. The form is up to you.

4. Training-Ready Output

Export the reviewed dataset in a structured form ready for downstream training. Format is your choice — document why you picked it.

Pipeline Service

- Runnable end-to-end
- Long-running stages (synthesis especially) should be handled sensibly — async jobs, batching, checkpointing, or equivalent
- Configuration externalized
- Intermediate artifacts observable (manifests, logs, samples)

Advanced (Optional, High-Impact)

- Awareness of how synthetic data can mislead a downstream model, and how your pipeline addresses it
- Automated quality signals feeding into review
- Handling of edge cases in the source text
- Caching, retries, resumability for long-running stages
- Tests for critical logic

Deliverables

- Source code
- README covering pipeline architecture, model and prompt-source choices, review approach, output format, observed quality issues, trade-offs, and limitations
- A small sample of the final dataset (a few dozen examples is fine) demonstrating the output format

We value clarity and judgment over completeness.

How We Evaluate

- Sound pipeline design
- Quality and diversity of generated prompts
- Thoughtfulness of the review approach
- Practical awareness of Egyptian Arabic and TTS limitations
- Reliability of long-running stages
- Clarity of the final output format and documentation